Introduction to Digital Logic Design Lab
EECS 31L

Lab #05

Student Name: STEVEN KUZHIPALA

Student ID #: 51378962

Date: 3/11/26

Datapath of RISC-V Single Cycle Processor

1. Design & Reasoning of the Complete Processor:

In this lab, we finish designing the Single cycle processor by creating a ALU controller module (to control the specific operation being performed by the ALU, after getting the instruction type coming from the Controller) , and the Controller module (to choose whether to use the register or immediate value as operands, to enable writing to a register or data memory, to also feed the ALUOp signal into the ALU Controller, to indicate the type of instruction for the ALU Controller to determine the appropriate operation).

The design of the RISC-V processor, with the ALU Controller and the Controller is based on the image below:

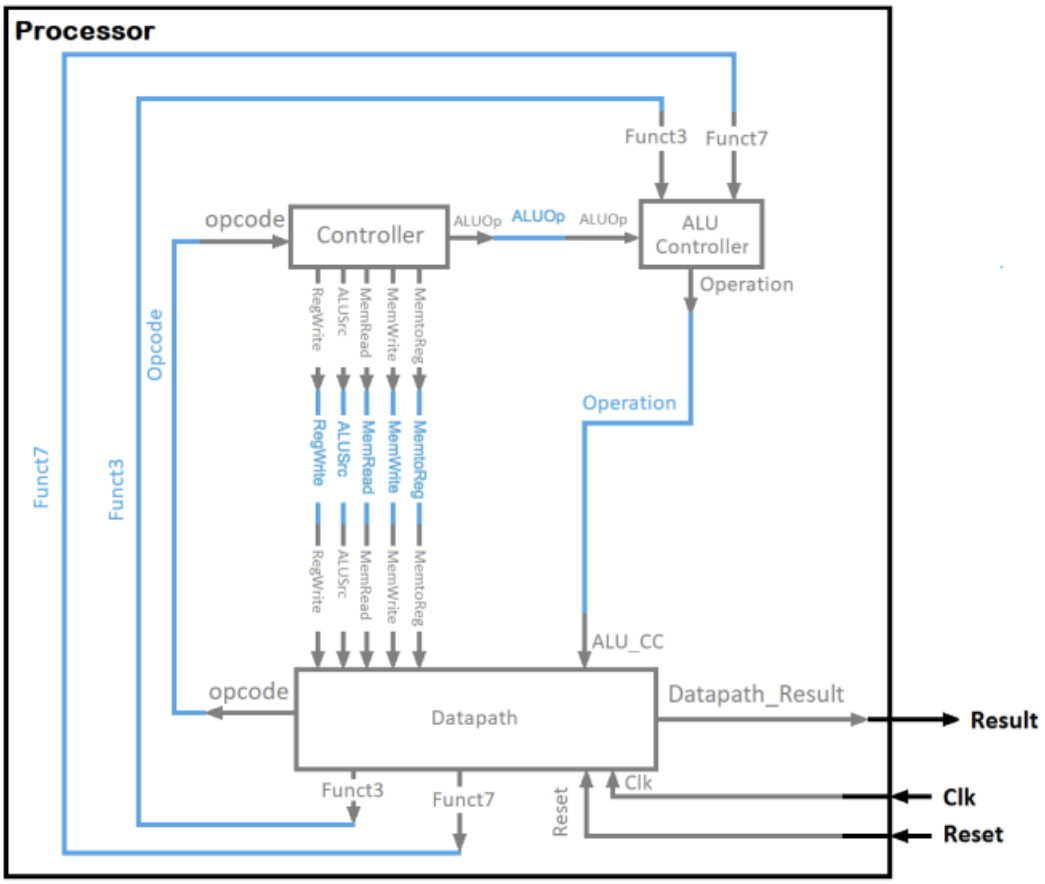
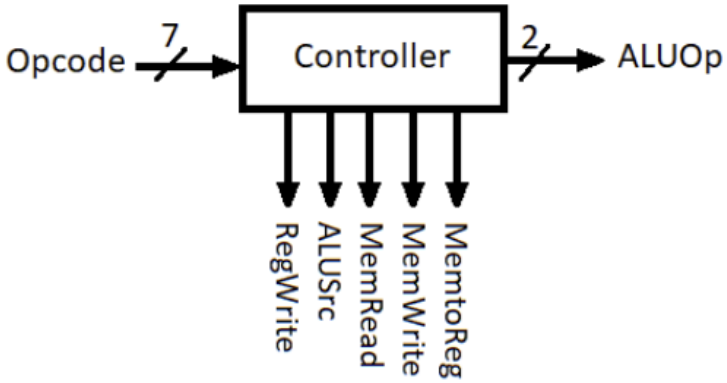


Figure 1: Processor.

2. The Controller Module:

The Controller module is used to choose whether to use the register or immediate value as operands, to enable writing to a register or data memory, to also feed the ALUOp signal into the ALU Controller, to indicate the type of instruction for the ALU Controller to determine the appropriate operation.

The design of the Controller module is based of the image given below:



Since the 6 output signals of the controller are solely dependent on a specific value in the 1input signal line (the Opcode), we can refer to the table given below and break down the logic of assigning the output wires with the 4 possible Opcode input combinations.

Table 2 : Control Signals.

		Opcode			
		0110011	0010011	0000011	0100011
		AND, OR, ADD, SUB, SLT, NOR	ANDI, ORI, ADDI, SLTI, NORI	LW	SW
Control Signals	MemtoReg	0	0	1	0
	MemWrite	0	0	0	1
	MemRead	0	0	1	0
	ALUSrc	0	1	1	1
	Regwrite	1	1	1	0
	ALUOp	10	00	01	01

To design this module, I created 4 different wires to signify the 4 possible opcode combinations to reduce any errors from typing the 7 bit opcodes wrong, and to also make it clearer to check the values as well.

I then used the Conditional operator (ternary operator) to easily assign the output signals values as below, making sure to set the first value for the output wire to be equal to the value that occurs the most in the Control Signal:

```
assign MemtoReg = (Opcode == Opcode_3) ? 1'b1 : 1'b0;

assign MemWrite = (Opcode == Opcode_4) ? 1'b1 : 1'b0;

assign MemRead  = (Opcode == Opcode_3) ? 1'b1 : 1'b0;

assign ALUSrc   = (Opcode == Opcode_1) ? 1'b0 : 1'b1;

assign RegWrite = (Opcode == Opcode_4) ? 1'b0 : 1'b1;

assign ALUOp    = (Opcode == Opcode_1)? 2'b10 : // R-type
                  (Opcode == Opcode_2)? 2'b00 : // I-type
                  (Opcode == Opcode_3 || Opcode == Opcode_4) ? 2'b01 : // LW and SW
                  2'b00; // Default
```

3. The ALU Controller Module:

The Controller module is used to control the specific operation being performed by the ALU, after getting the instruction type coming from the Controller. The ALU controller takes in the Funct3, Funct7, and the ALUOp signals, which are then used for generating the specific operation the ALU has to perform.

The ALUOp differentiates between R-type, I-type, and LW and SW functions.

The Funct3 signal differentiates between ADD/ADDI/SUB, OR/ORI, AND/ANDI, SLT/SLTI/LW/SW, NOR/NORI operations.

The Funct7 signal only differentiates between the ADD and SUB operations.

The design of the ALU Controller module is based of the image given below:

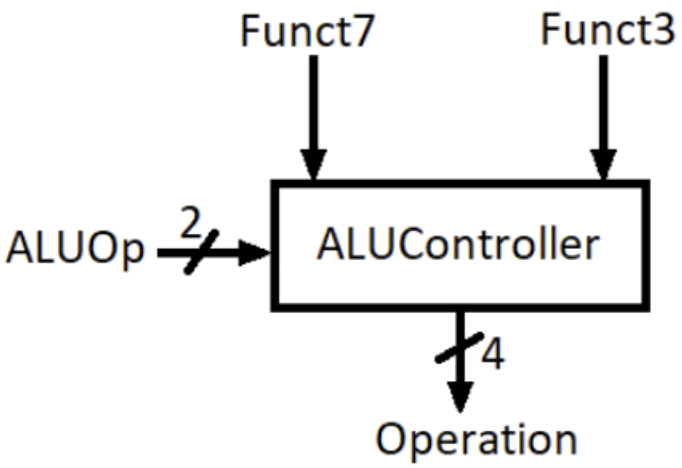


Figure 3: ALUController.

Similar to the Controller module, we can refer to the table given below, and break down the logic of assigning the output wires with the 3 input signal the ALU controller module receives:

Table 4 : The truth table for the 4 operation code.

	Funct7	Funct3	ALUop	Operation			
AND	0000000	111	10	0	0	0	0
OR	0000000	110	10	0	0	0	1
NOR	0000000	100	10	1	1	0	0
SLT	0000000	010	10	0	1	1	1
ADD	0000000	000	10	0	0	1	0
SUB	0100000	000	10	0	1	1	0
ANDI	-	111	00	0	0	0	0
ORI	-	110	00	0	0	0	1
NORI	-	100	00	1	1	0	0
SLTI	-	010	00	0	1	1	1
ADDI	-	000	00	0	0	1	0
LW	-	010	01	0	0	1	0
SW	-	010	01	0	0	1	0

To design this module, I identified the pattern of combinators that lead to positive signals on separate Operation indexes from 0-3. This way it was easier to assign a certain index a 1 or 0 signal based on the 3 input signals.

Assignment of operation index 0 was done in lab manual itself.

```
assign Operation [0]=
 ( ( (Funct3==3'b110) ||  ( (Funct3==3'b010) && (ALUOp[0]==1'b0) ) ) )  ? 1'b1: 1'b0;

assign Operation[1] =
    ((Funct3 == 3'b000) || (Funct3 == 3'b010)) ? 1'b1 : 1'b0;

assign Operation[2] =
    ((Funct3 == 3'b100) ||
     ((Funct3 == 3'b010) && (ALUOp[0] == 1'b0)) ||
     ((Funct7 == 7'b0100000) && (ALUOp[1] == 1'b1)))
    ? 1'b1 : 1'b0;

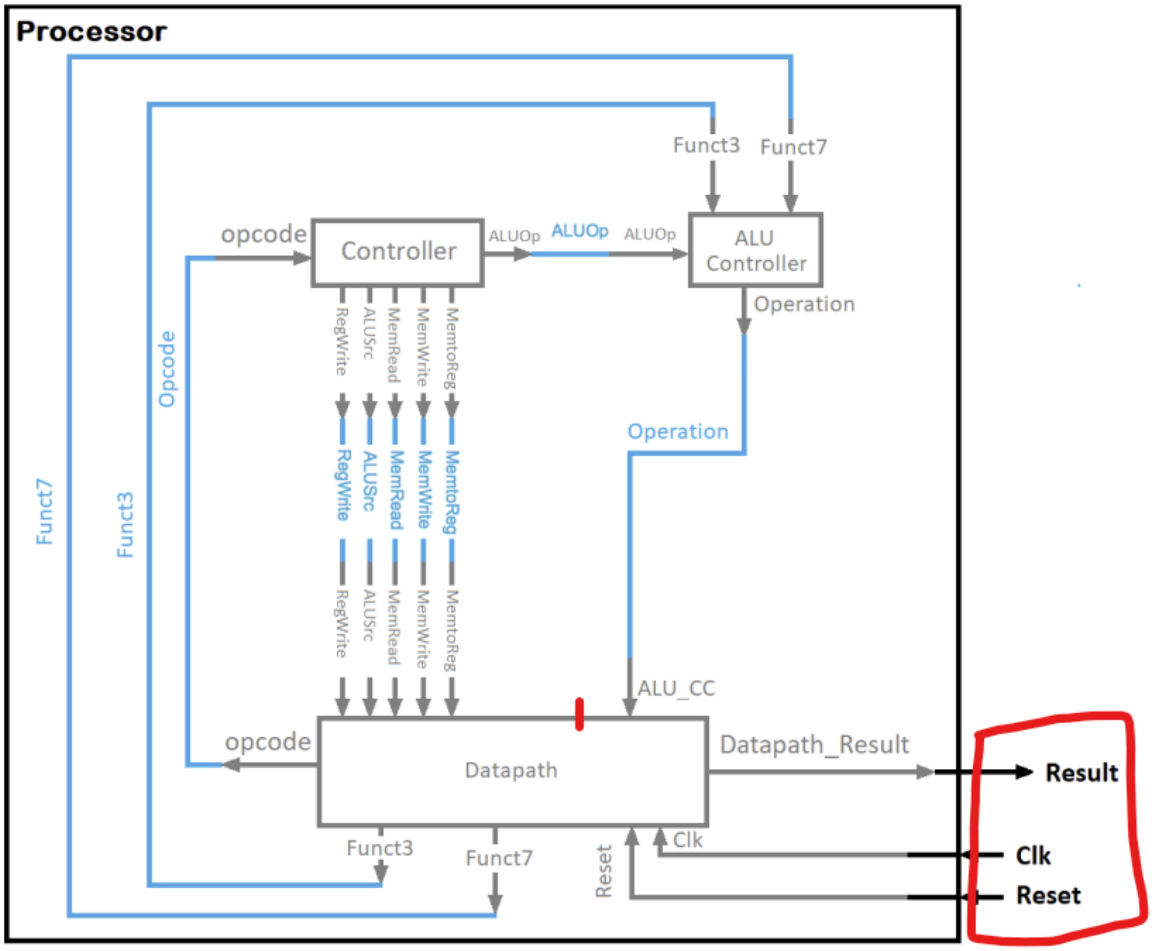
assign Operation[3] =
    (Funct3 == 3'b100)
     ? 1'b1 : 1'b0;
```

4. The Processor Module:

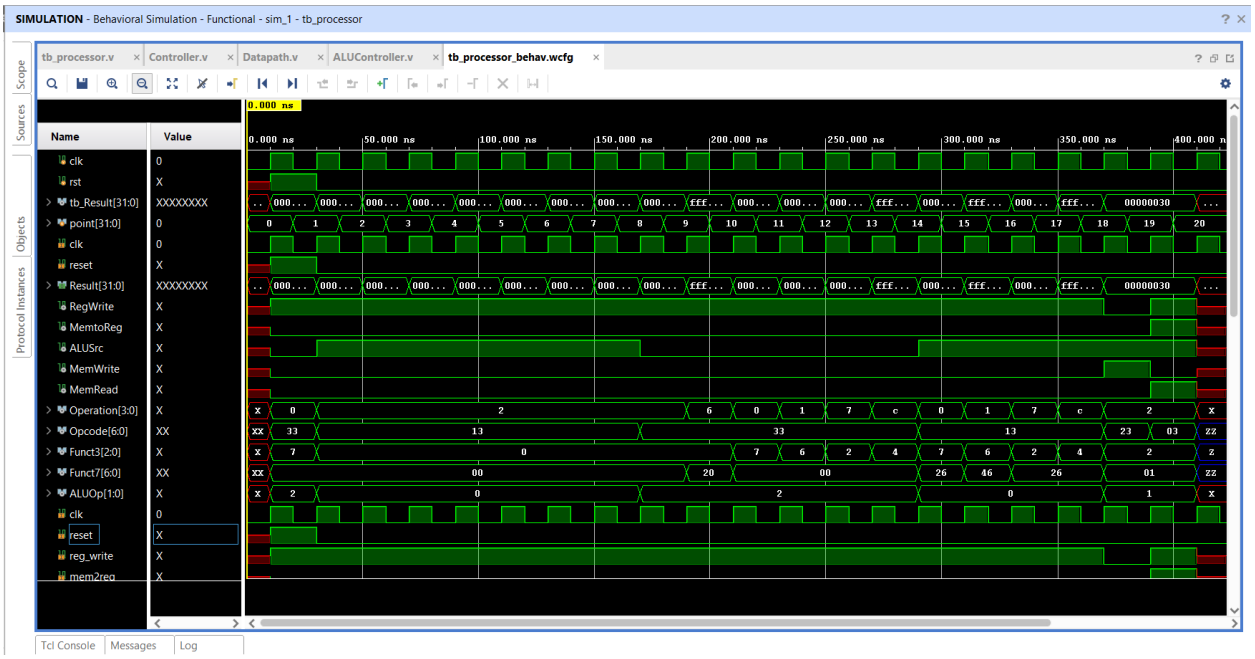
This module dealt with connecting the Controller and ALU controller modules, including the creation of wires that connect the modules and the Processor as well.

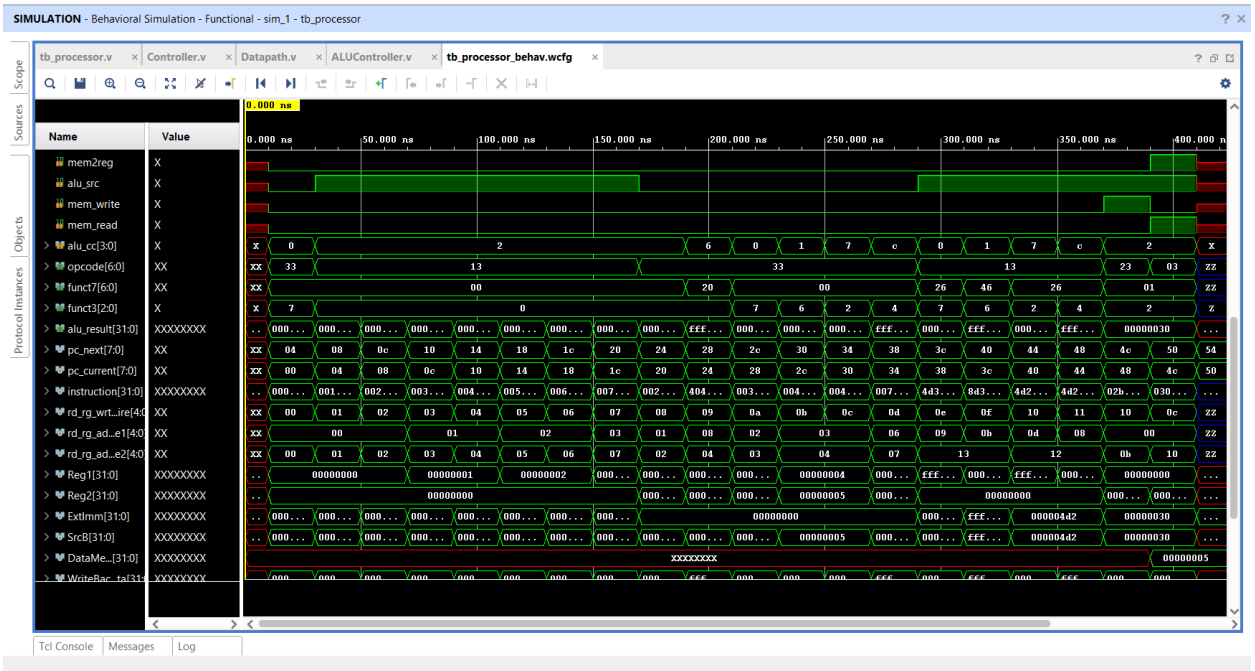
The processor solely depends on the two input lines, the clock and the reset lines, and

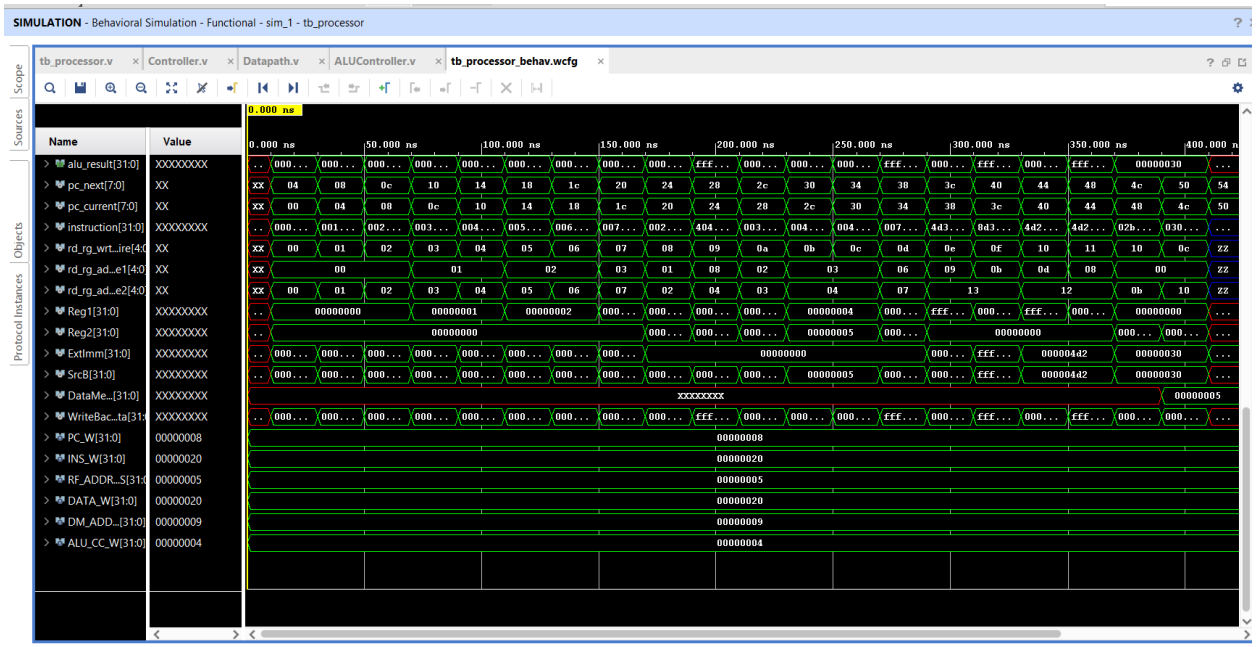
has one output line Result as marked below:



Creating this processor was a similar process to creating instants in the Datapath module, using our various modules designed before.


SIMULATION - Behavioral Simulation - Functional - sim_1 - tb_processor
tb_processor.v
Controller.v
Datapath.v
ALUController.v
tb_processor_behav.wcfg
Name
Value
clk
rst
tb_Result[31:0]
point[31:0]
clk
reset
Result[31:0]
RegWrite
MemtoReg
ALUSrc
MemWrite
MemRead
Operation[3:0]
Opcode[6:0]
Funct3[2:0]
Funct7[6:0]
ALUOp[1:0]
clk
reset
reg_write
mem2reg
Tcl Console
Messages
Log

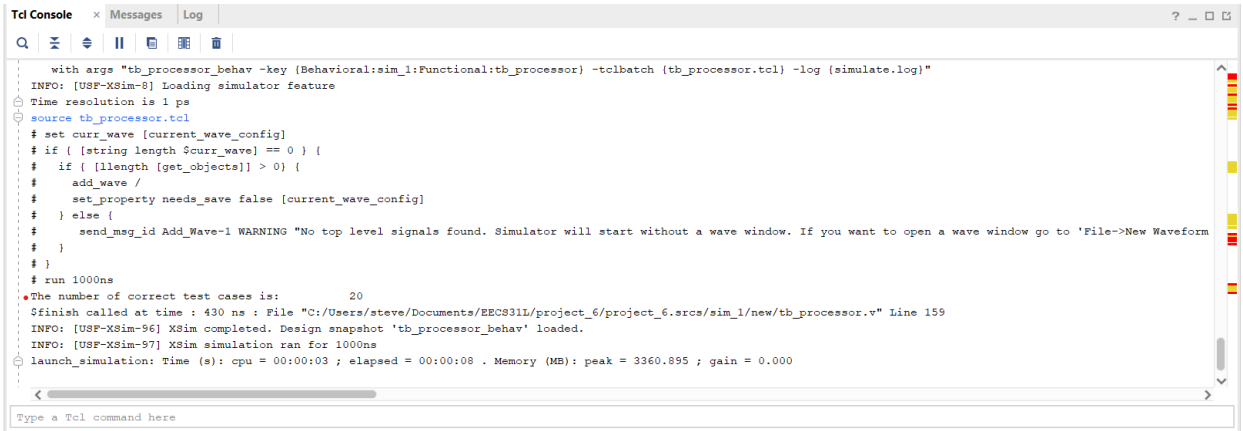

SIMULATION - Behavioral Simulation - Functional - sim_1 - tb_processor
tb_processor.v
Controller.v
Datapath.v
ALUController.v
tb_processor_behav.wcfg
Name
Value
mem2reg
alu_src
mem_write
mem_read
alu_cc[3:0]
opcode[6:0]
funct7[6:0]
funct3[2:0]
alu_result[31:0]
pc_next[7:0]
pc_current[7:0]
instruction[31:0]
Reg1[31:0]
Reg2[31:0]
ExtImm[31:0]
SrcB[31:0]
Tcl Console
Messages
Log



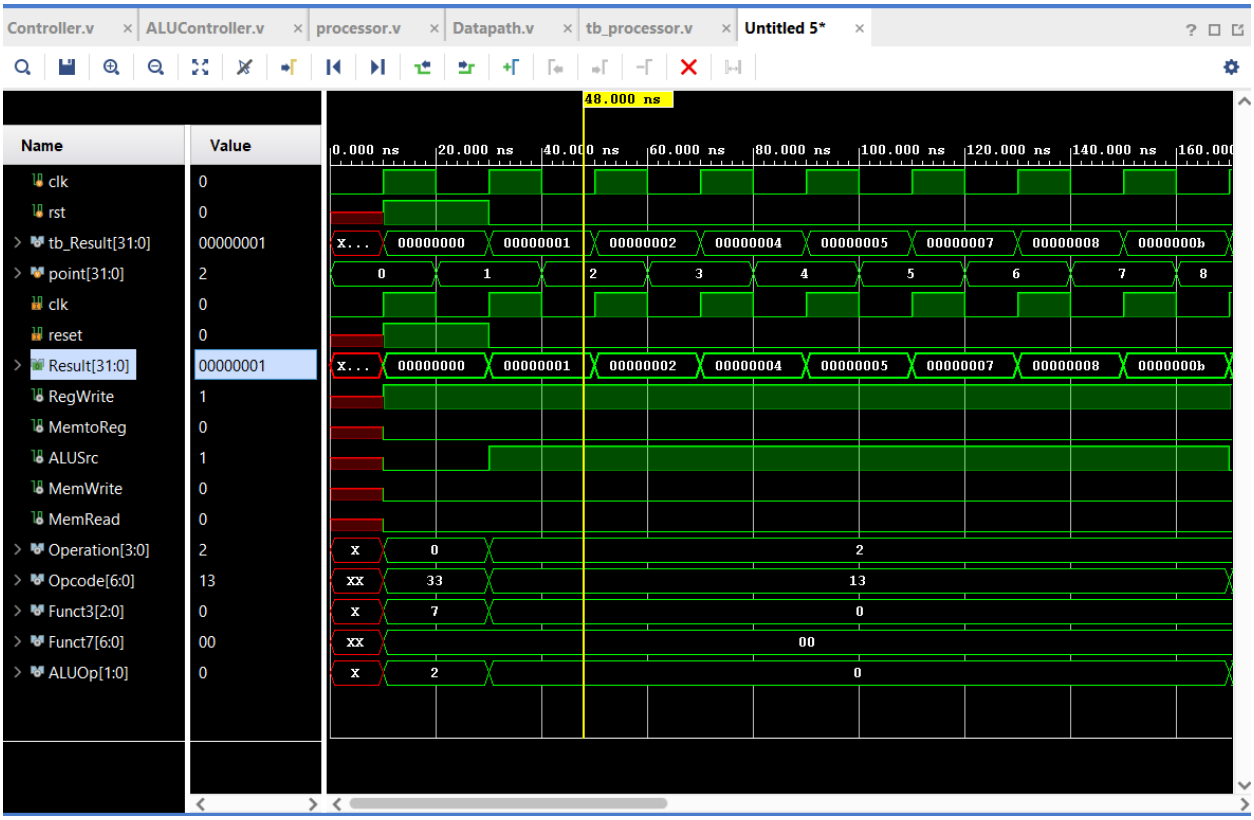
4.Testing and Simulation Verification:

Waveforms:

TCL Console output, showing the 20 test cases that have been passed.



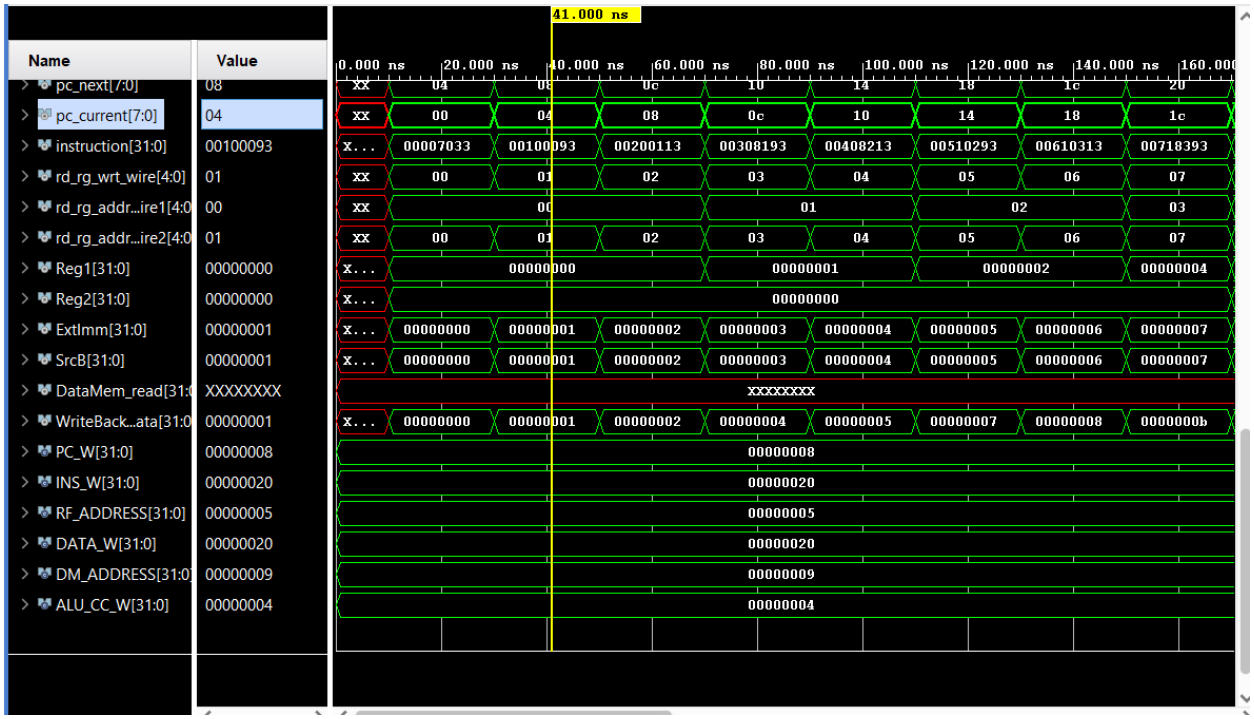
Annotating a particular result on the waveform:



IN the rising of the clock right before the yellow time line , the Opcode shows 2 (or 0000011) indicating the ADDI instruction to the controller.

The controller then recognizes the I-type instruction, sets the ALUSrc line to 1(the use the immediate value) , the RegWrite line to 1 (to write the result back) , and sets the ALU Op to 0, to allow the operation to be ADD for the ALU.

The ALU then computes the **register value 0 (on Reg1)** along with the **immediate value of 1 (on ExtImm)**, as shown in the extended screenshot below, and **puts the result as 1 to the 31-bit result line.**


41.000 ns
Name
Value
0.000 ns
20.000 ns
40.000 ns
60.000 ns
80.000 ns
100.000 ns
120.000 ns
140.000 ns
pc_next[7:0]
08
pc_current[7:0]
04
instruction[31:0]
00100093
rd_rg_wrt_wire[4:0]
01
rd_rg_addr...ire1[4:0]
00
rd_rg_addr...ire2[4:0]
01
Reg1[31:0]
00000000
Reg2[31:0]
00000000
ExtImm[31:0]
00000001
SrcB[31:0]
00000001
DataMem_read[31:0]
XXXXXXXX
WriteBack...ata[31:0]
00000001
PC_W[31:0]
00000008
INS_W[31:0]
00000020
RF_ADDRESS[31:0]
00000005
DATA_W[31:0]
00000020
DM_ADDRESS[31:0]
00000009
ALU_CC_W[31:0]
00000004